1. DATABASE COMMANDS

(DDL – Data Definition Language)

1.1 CREATE DATABASE

Definition: Creates a new database.

Syntax:

CREATE DATABASE database_name;

Example:

CREATE DATABASE school;

1.2 SHOW DATABASES

Definition: Lists all databases on the MySQL server.

Example:

SHOW DATABASES;

1.3 USE

Definition: Selects a database to work with.

Example:

USE school;

1.4 DROP DATABASE

Definition: Deletes a database permanently.

Example:

```
DROP DATABASE school;
```

Imp: This deletes all tables and data.

2. TABLE COMMANDS (DDL)

2.1 CREATE TABLE

Definition: Creates a new table.

Syntax:

```
CREATE TABLE table_name (
    column_name datatype,
    column_name datatype
);
```

Example:

```
CREATE TABLE students (
    id INT,
    name VARCHAR(50),
    age INT
);
```

2.2 DESCRIBE (DESC)

Definition: Shows table structure.

Example:

DESC students;

2.3 DROP TABLE

Definition: Deletes a table.

Example:

DROP TABLE students;

2.4 TRUNCATE TABLE

Definition: Deletes all records but keeps the table.

Example:

TRUNCATE TABLE students;

2.5 ALTER TABLE

Definition: Modifies an existing table.

Add a column

ALTER TABLE students ADD email VARCHAR(100);

Modify a column

ALTER TABLE students MODIFY age TINYINT;

Drop a column

ALTER TABLE students DROP email;

3. DATA MANIPULATION COMMANDS (DML)

3.1 INSERT

Definition: Adds records to a table.

Insert single row

INSERT INTO students VALUES (1, 'Alice', 20);

Insert specific columns

INSERT INTO students (id, name) VALUES (2, 'Bob');

3.2 SELECT

Definition: Retrieves data from a table.

Select all columns

SELECT * FROM students;

Select specific columns

SELECT name, age FROM students;

3.3 UPDATE

Definition: Modifies existing records.

Example:

```
UPDATE students
SET age = 21
WHERE id = 1;
```

Imp: Without WHERE, all rows will be updated.

3.4 DELETE

Definition: Removes records from a table.

```
DELETE FROM students WHERE id = 2;
```

Imp: Without WHERE, all rows will be deleted.

4. CONSTRAINTS (RULES ON DATA)

4.1 PRIMARY KEY

Definition: Uniquely identifies each row.

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50)
);
```

4.2 AUTO_INCREMENT

Definition: Automatically increases numeric values.

```
id INT PRIMARY KEY AUTO_INCREMENT
```

4.3 NOT NULL

Definition: Prevents NULL values.

name VARCHAR(50) NOT NULL

4.4 UNIQUE

Definition: Ensures unique values.

email VARCHAR(100) UNIQUE

4.5 DEFAULT

Definition: Sets a default value.

age INT DEFAULT 18

4.6 FOREIGN KEY

Definition: Links two tables.

```
CREATE TABLE marks (
    student_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

5. FILTERING & CONDITIONS (WHERE CLAUSE)

5.1 WHERE

SELECT * FROM students WHERE age > 18;

5.2 AND / OR / NOT

SELECT * FROM students WHERE age > 18 AND name = 'Alice';

5.3 IN

SELECT * FROM students WHERE age IN (18, 20, 22);

5.4 BETWEEN

SELECT * FROM students WHERE age BETWEEN 18 AND 22;

5.5 LIKE (Pattern Matching)

SELECT * FROM students WHERE name LIKE 'A%';

Pattern	Meaning
%	Any number of characters
_	Single character

5.6 IS NULL

SELECT * FROM students WHERE age IS NULL;

6. SORTING & LIMITING RESULTS

6.1 ORDER BY

SELECT * FROM students ORDER BY age DESC;

6.2 LIMIT

SELECT * FROM students LIMIT 5;

7. AGGREGATE FUNCTIONS

Function	Description
COUNT()	Number of rows
SUM()	Total
AVG()	Average
MIN()	Minimum
MAX()	Maximum

Example:

SELECT COUNT(*) FROM students;

8. GROUPING DATA

8.1 GROUP BY

SELECT age, COUNT(*)
FROM students
GROUP BY age;

8.2 HAVING

Definition: Filters grouped data.

```
SELECT age, COUNT(*)
FROM students
GROUP BY age
HAVING COUNT(*) > 1;
```

9. JOINS (MULTIPLE TABLES)

9.1 INNER JOIN

```
SELECT students.name, marks.score
FROM students
INNER JOIN marks
ON students.id = marks.student_id;
```

9.2 LEFT JOIN

```
SELECT students.name, marks.score
FROM students
LEFT JOIN marks
ON students.id = marks.student_id;
```

9.3 RIGHT JOIN

```
SELECT students.name, marks.score
FROM students
```

```
RIGHT JOIN marks
ON students.id = marks.student_id;
```

10. SUBQUERIES

Definition: Query inside another query.

```
SELECT name
FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

11. INDEXES

Definition: Improves search performance.

```
CREATE INDEX idx_name ON students(name);
```

12. VIEWS

Definition: Virtual table based on a query.

```
CREATE VIEW adult_students AS
SELECT * FROM students WHERE age >= 18;
```

13. TRANSACTIONS (TCL)

COMMIT

```
COMMIT;
```

ROLLBACK

ROLLBACK;

SAVEPOINT

SAVEPOINT sp1;

14. COMMON SQL KEYWORDS SUMMARY

Keyword	Purpose
SELECT	Fetch data
INSERT	Add data
UPDATE	Modify data
DELETE	Remove data
CREATE	Create object
DROP	Delete object
ALTER	Modify structure
WHERE	Filter rows
JOIN	Combine tables
GROUP BY	Group rows
ORDER BY	Sort
LIMIT	Restrict rows

LAB PROGRAM 1

Consider the following schema for a Library Database:BOOK (Book_id, Title, Publisher_Name, Pub_Year)

BOOK_AUTHORS (Book_id, Author_Name)

PUBLISHER (Name, Address, Phone) BOOK_COPIES (Book_id, Branch_id, Noof_Copies)

BOOK_LENDING (Book_id, Branch_id, Card_No, Date_Out, Due_Date)LIBRARY_PROGRAMME (Branch_id, Branch_Name,Address) CARD (Card_No)

Write SQL queries to

a. Retrieve details of all books in the library – id, title, name of publisher, authors,number ofc opies in each Programme, etc.

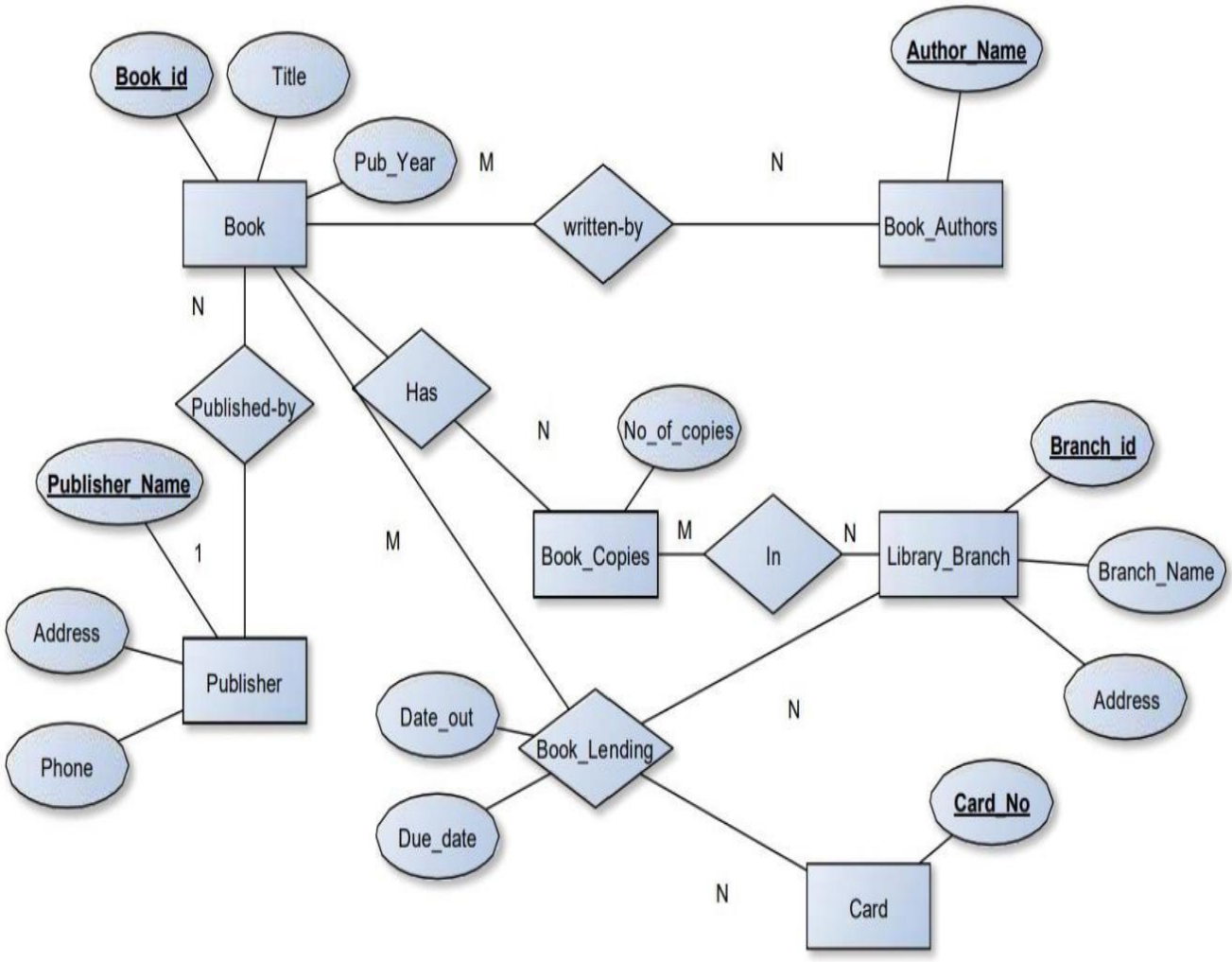
b. Get the particulars of borrowers who have borrowed more than 2 books, in the year 2020.

c. Delete a book in BOOK table. Update the contents of other tables to reflect this data manipulation operation.

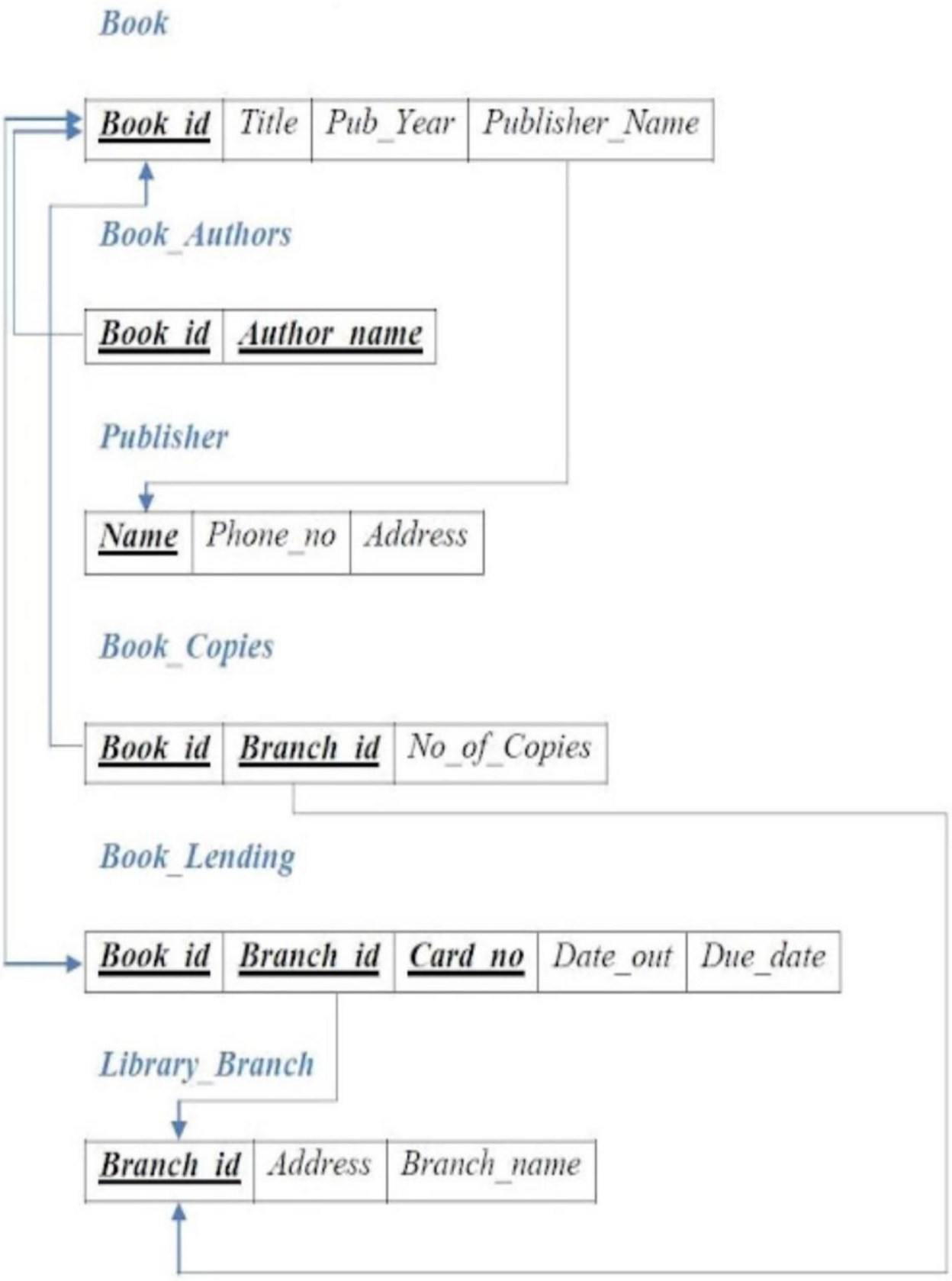
d. Display the total number of books published by each Publisher.

e. Create a view of all books and its number of copies that are currently available in the Library.

ER DIAGRAM



SCHEMA DIAGRAM



1. CREATE DATABASE

```
CREATE DATABASE LibraryDB;
USE LibraryDB;
```

2. TABLE CREATION

PUBLISHER

```
CREATE TABLE PUBLISHER (
    NAME VARCHAR(20) PRIMARY KEY,
    PHONE BIGINT,
    ADDRESS VARCHAR(20)
);
```

BOOK

```
CREATE TABLE BOOK (
    BOOK_ID INT PRIMARY KEY,
    TITLE VARCHAR(20),
    PUB_YEAR VARCHAR(20),
    PUBLISHER_NAME VARCHAR(20),
    FOREIGN KEY (PUBLISHER_NAME) REFERENCES PUBLISHER(NAME)
    ON DELETE CASCADE
);
```

BOOK_AUTHORS

```
CREATE TABLE BOOK_AUTHORS (
    AUTHOR_NAME VARCHAR(20),
    BOOK_ID INT,
    PRIMARY KEY (BOOK_ID, AUTHOR_NAME),
    FOREIGN KEY (BOOK_ID) REFERENCES BOOK(BOOK_ID)
    ON DELETE CASCADE
);
```

LIBRARY_BRANCH

```
CREATE TABLE LIBRARY_BRANCH (
    BRANCH_ID INT PRIMARY KEY,
    BRANCH_NAME VARCHAR(50),
    ADDRESS VARCHAR(50)
);
```

BOOK_COPIES

```
CREATE TABLE BOOK_COPIES (
    NO_OF_COPIES INT,
    BOOK_ID INT,
    BRANCH_ID INT,
    PRIMARY KEY (BOOK_ID, BRANCH_ID),
    FOREIGN KEY (BOOK_ID) REFERENCES BOOK(BOOK_ID)
    ON DELETE CASCADE,
    FOREIGN KEY (BRANCH_ID) REFERENCES LIBRARY_BRANCH(BRANCH_ID)
    ON DELETE CASCADE
);
```

CARD

```
CREATE TABLE CARD (
    CARD_NO INT PRIMARY KEY
);
```

BOOK_LENDING

```
CREATE TABLE BOOK_LENDING (
    DATE_OUT DATE,
    DUE_DATE DATE,
    BOOK_ID INT,
    BRANCH_ID INT,
```

```
    CARD_NO INT,
    PRIMARY KEY (BOOK_ID, BRANCH_ID, CARD_NO),
    FOREIGN KEY (BOOK_ID) REFERENCES BOOK(BOOK_ID)
    ON DELETE CASCADE,
    FOREIGN KEY (BRANCH_ID) REFERENCES LIBRARY_BRANCH(BRANCH_ID)
    ON DELETE CASCADE,
    FOREIGN KEY (CARD_NO) REFERENCES CARD(CARD_NO)
    ON DELETE CASCADE
);
```

3. INSERT VALUES

PUBLISHER

```
INSERT INTO PUBLISHER VALUES
('MCGRAW-HILL', 9989076587, 'BANGALORE');
INSERT INTO PUBLISHER VALUES
('PEARSON', 9889076565, 'KOLKATTA');
INSERT INTO PUBLISHER VALUES
('SPRINGER', 9989076544, 'PUNE');
INSERT INTO PUBLISHER VALUES
('PENGUIN', 9889056565, 'GUJARATH');
INSERT INTO PUBLISHER VALUES
('TATA', 9889073560, 'GOA');
```

BOOK

```
INSERT INTO BOOK VALUES
(1, 'DBMS', 'JAN-2017', 'MCGRAW-HILL');
INSERT INTO BOOK VALUES
(2, 'JAVA', 'JUNE-2017', 'MCGRAW-HILL');
```

```
INSERT INTO BOOK VALUES
(3, 'SE', 'JAN-2017', 'PEARSON');
INSERT INTO BOOK VALUES
(4, 'PYTHON', 'JAN-2017', 'PENGUIN');
INSERT INTO BOOK VALUES
(5, 'ADBMS', 'JAN-2017', 'TATA');
```

B**OOK_AUTHORS**

```
INSERT INTO BOOK_AUTHORS VALUES
('NAVATHE', 1);
INSERT INTO BOOK_AUTHORS VALUES
('GALVIN', 2);
INSERT INTO BOOK_AUTHORS VALUES
('SOMERVILLE', 3);
INSERT INTO BOOK_AUTHORS VALUES
('IVAN', 4);
INSERT INTO BOOK_AUTHORS VALUES
('BALGURU', 5);
```

LIBRARY_BRANCH

```
INSERT INTO LIBRARY_BRANCH VALUES
(10, 'RR NAGAR', 'BANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES
(11, 'NITTE', 'BANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES
(12, 'MIT', 'MANIPAL');
INSERT INTO LIBRARY_BRANCH VALUES
(13, 'MUDBIDRI', 'MANGALORE');
INSERT INTO LIBRARY_BRANCH VALUES
(14, 'SMVT', 'UDUPI');
```

BOOK_COPIES

```
INSERT INTO BOOK_COPIES VALUES (10, 1, 10);
```

```
INSERT INTO BOOK_COPIES VALUES (5, 1, 11);
INSERT INTO BOOK_COPIES VALUES (11, 2, 12);
INSERT INTO BOOK_COPIES VALUES (6, 3, 13);
INSERT INTO BOOK_COPIES VALUES (12, 3, 14);
```

CARD

```
INSERT INTO CARD VALUES (101);
INSERT INTO CARD VALUES (102);
INSERT INTO CARD VALUES (103);
INSERT INTO CARD VALUES (104);
INSERT INTO CARD VALUES (105);
```

BOOK_LENDING

```
INSERT INTO BOOK_LENDING VALUES
('2020-01-01', '2020-06-01', 1, 10, 101);
INSERT INTO BOOK_LENDING VALUES
('2020-02-01', '2020-06-01', 1, 11, 101);
INSERT INTO BOOK_LENDING VALUES
('2020-01-01', '2020-12-06', 2, 12, 102);
INSERT INTO BOOK_LENDING VALUES
('2020-04-01', '2020-06-01', 3, 10, 103);
INSERT INTO BOOK_LENDING VALUES
('2020-01-01', '2020-06-01', 1, 13, 101);
```

SQL QUERIES

1. Retrieve details of all books in the library

```
SELECT
B.BOOK_ID,
B.TITLE,
B.PUBLISHER_NAME,
A.AUTHOR_NAME,
C.NO_OF_COPIES,
```

```
L.BRANCH_ID
FROM BOOK B
JOIN BOOK_AUTHORS A ON B.BOOK_ID = A.BOOK_ID
JOIN BOOK_COPIES C ON B.BOOK_ID = C.BOOK_ID
JOIN LIBRARY_BRANCH L ON C.BRANCH_ID = L.BRANCH_ID;
```

BOOK_ID	TITLE	PUBLISHER_NAME	AUTHOR_NAME	NO_OF_COPIES	BRANCH_ID
1	DBMS	MCGRAW-HILL	NAVATHE	10	10
1	DBMS	MCGRAW-HILL	NAVATHE	5	11
2	JAVA	MCGRAW-HILL	GALVIN	11	12
3	SE	PEARSON	SOMERVILLE	6	13
3	SE	PEARSON	SOMERVILLE	12	14

2. Borrowers who borrowed more than 2 books in 2020

```
SELECT CARD_NO
FROM BOOK_LENDING
WHERE DATE_OUT BETWEEN '2020-01-01' AND '2020-12-31'
GROUP BY CARD_NO
HAVING COUNT(*) > 2;
```

CARD_NO
101

3. Delete a book and update other tables

```
DELETE FROM BOOK WHERE BOOK_ID = 3;
```

4. Display total number of books published by each publisher

```
SELECT Publisher_Name, COUNT(*) AS Total_Books
FROM BOOK
GROUP BY Publisher_Name;
```

PUBLISHER_NAME	TOTAL_BOOKS
MCGRAW-HILL	2
PENGUIN	1
TATA	1

5. Create a view of all books and available copies

```
CREATE VIEW V_BOOKS AS
SELECT
B.BOOK_ID,
B.TITLE,
SUM(C.NO_OF_COPIES) AS TOTAL_COPIES
FROM BOOK B
JOIN BOOK_COPIES C ON B.BOOK_ID = C.BOOK_ID
GROUP BY B.BOOK_ID, B.TITLE;

SELECT * FROM V_BOOKS;
```

PROGRAM 2

Consider the schema for Company Database:

EMPLOYEE (SSN, Name, Address, Gender, Salary, SuperSSN,DNo)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

DLOCATION (DNo,DLoc)

PROJECT (PNo, PName, PLocation, DNo)

WORKS_ON (SSN, PNo, Hours)

Write SQL queries for the following

a. Convert employee name into uppercase whenever an employee record is inserted or updated. Trigger to fire before the insert or update.

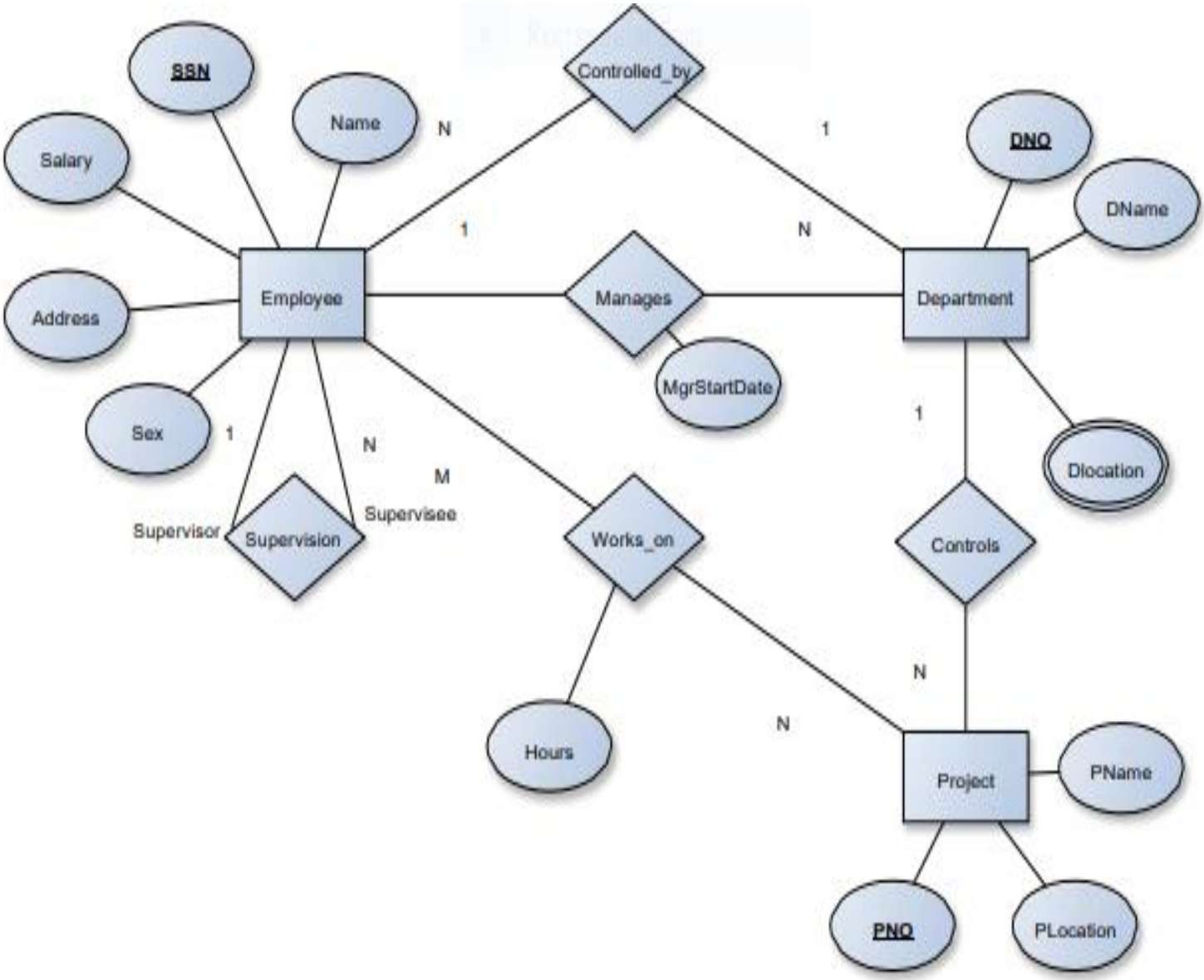
b. Make a list of all project numbers for projects that involve an employee whose last name is 'Scott', either as a worker or as a manager of the department that controls the project.

c. Show the resulting salaries if every employee working on the 'IoT' project is given a 10 percent raise.

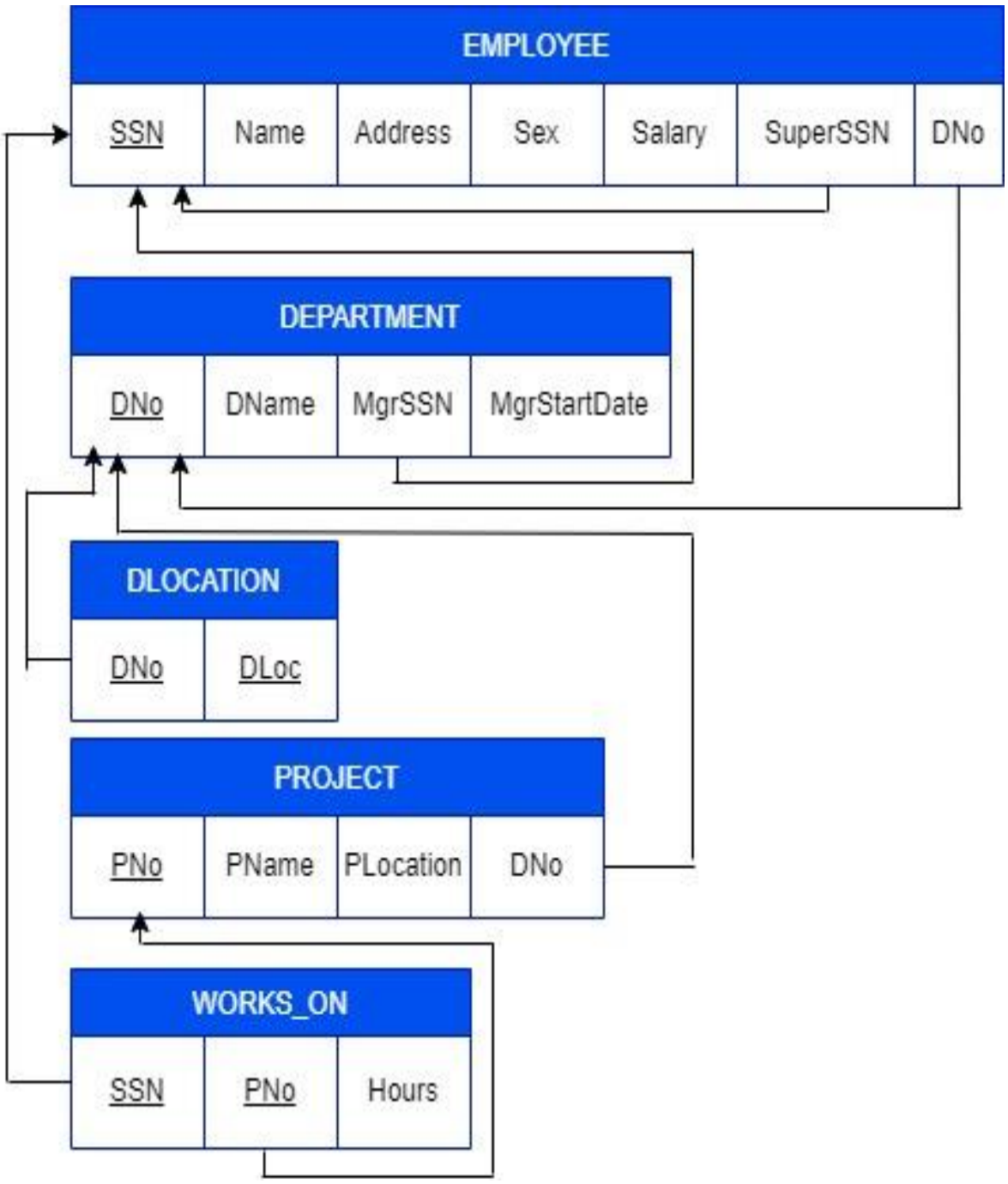
d. Find the sum of the salaries of all employees of the 'Accounts' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

e. Retrieve the name of each employee who works on all the projects controlled by department number 5.

ER DIAGRAM


SSN
Name
Salary
Address
Sex
Employee
Controlled_by
N
1
1
N
DNO
DName
Department
Manages
MgrStartDate
1
Dlocation
1
N
M
Supervisor
Supervision
Supervisee
Works_on
Controls
N
N
Hours
Project
PName
PNO
PLocation

SCHEMA DIAGRAM



1. Table Creation

#EMPLOYEE Table

```
CREATE TABLE EMPLOYEE (
   SSN BIGINT PRIMARY KEY,
   NAME VARCHAR(20),
   ADDRESS VARCHAR(30),
   GENDER CHAR(1),
   SALARY DECIMAL(10,2),
   SUPERSSN BIGINT,
   DNO INT,
   FOREIGN KEY (SUPERSSN) REFERENCES EMPLOYEE(SSN) ON DELETE CASCADE
);
```

#DEPARTMENT Table

```
CREATE TABLE DEPARTMENT (
   DNO INT PRIMARY KEY,
   DNAME VARCHAR(20),
   MGRSSN BIGINT,
   MGRSTARTDATE DATE,
   FOREIGN KEY (MGRSSN) REFERENCES EMPLOYEE(SSN) ON DELETE CASCADE
);
```

#Add Department Number to EMPLOYEE Table

```
ALTER TABLE EMPLOYEE
```

```
ADD CONSTRAINT FK_EMP_DEPT
FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNO)
ON DELETE CASCADE;
```

#DLOCATION Table

```
CREATE TABLE DLOCATION (
    DNO INT,
    DLOC VARCHAR(20),
    PRIMARY KEY (DNO, DLOC),
    FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNO) ON DELETE CASCADE
);
```

#PROJECT Table

```
CREATE TABLE PROJECT (
    PNO INT PRIMARY KEY,
    PNAME VARCHAR(20),
    PLOCATION VARCHAR(20),
    DNO INT,
    FOREIGN KEY (DNO) REFERENCES DEPARTMENT(DNO) ON DELETE CASCADE
);
```

#WORKS_ON Table

```
CREATE TABLE WORKS_ON (
    SSN BIGINT,
```

```
    PNO INT,
    HOURS INT,
    PRIMARY KEY (SSN, PNO),
    FOREIGN KEY (SSN) REFERENCES EMPLOYEE(SSN) ON DELETE CASCADE,
    FOREIGN KEY (PNO) REFERENCES PROJECT(PNO) ON DELETE CASCADE
);
```

TABLE: EMPLOYEE						
SSN	**NAME**	**ADDRESS**	**GENDER**	**SALARY**	**SUPERSSN**	**DNO**
123456789	Asha	Bangalore	F	50000	NULL	1
234567891	Sheela	Hassan	F	45000	123456789	2
345678912	Pallavi	Bangalore	F	60000	NULL	3
456789123	Shreyas	Mysore	M	120000	NULL	4
567891234	Mohan	Bangalore	M	18000	NULL	5
678912345	Scott	Bangalore	M	50000	NULL	6
789123456	Divya	Bangalore	F	650000	345678912	3
891234567	Sapna	Bangalore	F	50000	345678912	3
912345678	Revan	Hubli	M	10000	345678912	3
113489567	Alice	Mysore	F	650000	345678912	3
112456789	Manoj	Mysore	M	25000	345678912	3
101234567	John	Hubli	M	20000	456789123	4

TABLE:DEPARTMENT			
DNO	**DNAME**	**MGRSSN**	**MGRSTARTDATE**
1	CSE	123456789	01-JAN-2010
2	ISE	234567891	15-FEB-2011
3	ECE	345678912	01-MAR-2012
4	Accounts	456789123	15-APR-2013
5	Research	567891234	02-MAY-2014

6	AIML	678912345	15-JUN-2015

TABLE:DLOCATION	
DNO	**DLOCATION**
1	Bangalore
2	Hubli
3	Mysore
4	Hassan
5	Bangalore
6	Bangalore

TABLE:PROJECT			
PNO	**PNAME**	**PLOCATION**	**DNO**
11	Java	Bangalore	1
22	DotNet	Hassan	2
33	IOT	Bangalore	3
44	Android	Mysore	4
55	Big Data	Hubli	5
66	Web	Bangalore	6
77	ML	Mysore	5

TABLE:WORKS_ON		
SSN	**PNO**	**HOURS**
678912345	11	30
123456789	22	30
234567891	33	40
678912345	44	20
345678912	55	50
456789123	66	60
345678912	77	60

#Insert Employee

INSERT INTO EMPLOYEE (SSN, NAME, ADDRESS, GENDER, SALARY) VALUES

(123456789, 'Asha', 'Bangalore', 'F', 50000),

(234567891, 'Sheela', 'Hassan', 'F', 45000),

(345678912, 'Pallavi', 'Bangalore', 'F', 60000),

(456789123, 'Shreyas', 'Mysore', 'M', 120000),

(567891234, 'Mohan', 'Bangalore', 'M', 18000),

(678912345, 'Scott', 'Bangalore', 'M', 50000),

(789123456, 'Divya', 'Bangalore', 'F', 650000),

(891234567, 'Sapna', 'Bangalore', 'F', 50000),

(912345678, 'Revan', 'Hubli', 'M', 10000),

(113489567, 'Alice', 'Mysore', 'F', 650000),

(112456789, 'Manoj', 'Mysore', 'M', 25000),

(101234567, 'John', 'Hubli', 'M', 20000);

#INSERT DEPARTMENT

INSERT INTO DEPARTMENT (DNO, DNAME, MGRSSN, MGRSTARTDATE)

VALUES

(1, 'CSE', 123456789, '2010-01-01'),

(2, 'ISE', 234567891, '2011-02-15'),

(3, 'ECE', 345678912, '2012-03-01'),

(4, 'Accounts', 456789123, '2013-04-15'),

(5, 'Research', 567891234, '2014-05-02'),

(6, 'AIML', 678912345, '2015-06-15');

#Update Employee

UPDATE EMPLOYEE SET SUPERSSN = NULL, DNO = 1 WHERE SSN = 123456789;

UPDATE EMPLOYEE SET SUPERSSN = 123456789, DNO = 2 WHERE SSN = 234567891;

UPDATE EMPLOYEE SET SUPERSSN = NULL, DNO = 3 WHERE SSN = 345678912;

UPDATE EMPLOYEE SET SUPERSSN = NULL, DNO = 5 WHERE SSN = 567891234;

UPDATE EMPLOYEE SET SUPERSSN = NULL, DNO = 6 WHERE SSN = 678912345;

UPDATE EMPLOYEE SET SUPERSSN = 345678912, DNO = 3 WHERE SSN = 789123456;

UPDATE EMPLOYEE SET SUPERSSN = 345678912, DNO = 3 WHERE SSN = 891234567;

UPDATE EMPLOYEE SET SUPERSSN = 345678912, DNO = 3 WHERE SSN = 912345678;

UPDATE EMPLOYEE SET SUPERSSN = 345678912, DNO = 3 WHERE SSN = 113489567;

UPDATE EMPLOYEE SET SUPERSSN = 345678912, DNO = 3 WHERE SSN = 112456789;

UPDATE EMPLOYEE SET SUPERSSN = 456789123, DNO = 4 WHERE SSN = 101234567;

UPDATE EMPLOYEE SET SUPERSSN = NULL, DNO = 4

WHERE SSN = 456789123;

#INSERT DLOCATION

INSERT INTO DLOCATION (DNO, DLOC) VALUES
(1, 'Bangalore'),
(2, 'Hubli'),
(3, 'Mysore'),
(4, 'Hassan'),
(5, 'Bangalore'),
(6, 'Bangalore');

#INSERT PROJECT

INSERT INTO PROJECT (PNO, PNAME, PLOCATION, DNO) VALUES
(11, 'Java', 'Bangalore', 1),
(22, 'DotNet', 'Hassan', 2),
(33, 'IoT', 'Bangalore', 3),
(44, 'Android', 'Mysore', 4),
(55, 'Big Data', 'Hubli', 5),
(66, 'Web', 'Bangalore', 6),
(77, 'ML', 'Mysore', 5);

#INSERT WORKS_ON

INSERT INTO WORKS_ON (SSN, PNO, HOURS)VALUES
(678912345, 11, 30),
(123456789, 22, 30),

```
(234567891, 33, 40),
(678912345, 44, 20),
(345678912, 55, 50),
(456789123, 66, 60),
(345678912, 77, 60);
```

a. Convert employee name into uppercase whenever an employee record is inserted or updated. Trigger to fire before the insert or update

```
CREATE TRIGGER EMPLOYEE_NAME_UPPER
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
   SET NEW.NAME = UPPER(NEW.NAME);
END$$

CREATE TRIGGER EMPLOYEE_NAME_UPPER_UPDATE
BEFORE UPDATE ON EMPLOYEE
FOR EACH ROW
BEGIN
   SET NEW.NAME = UPPER(NEW.NAME);
END$$
```

b. Make a list of all project numbers for projects that involve an employee whose last name is 'Scott', either as a worker or as a manager of the department that controls the project.

```
SELECT DISTINCT P.PNO
```

```
FROM PROJECT P
JOIN DEPARTMENT D ON P.DNO = D.DNO
JOIN EMPLOYEE E ON D.MGRSSN = E.SSN
WHERE E.NAME = 'Scott'

UNION

SELECT DISTINCT P.PNO
FROM PROJECT P
JOIN WORKS_ON W ON P.PNO = W.PNO
JOIN EMPLOYEE E ON W.SSN = E.SSN
WHERE E.NAME = 'Scott';
```

PNO
11
44
66

c. Show the resulting salaries if every employee working on the ‘IoT’ project is given a 10 percent raise.

```
SELECT E.NAME,
    E.SALARY,
    (E.SALARY * 1.10) AS HIKE_SALARY
FROM EMPLOYEE E
JOIN WORKS_ON W ON E.SSN = W.SSN
JOIN PROJECT P ON W.PNO = P.PNO
WHERE P.PNAME = 'IoT';
```

ENAME	HIKE_SALARY
Sheela	49500

d.Find the sum of the salaries of all employees of the 'Accounts' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
SELECT
SUM(E.SALARY) AS TOTAL_SALARY,
MAX(E.SALARY) AS MAX_SALARY,
MIN(E.SALARY) AS MIN_SALARY,
AVG(E.SALARY) AS AVG_SALARY
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DNO = D.DNO
WHERE D.DNAME = 'Accounts';
```

SUM(E.SALARY)	MAX(E.SALARY)	MIN(E.SALARY)	AVG(E.SALARY)
140000	120000	20000	70000

e. Retrieve the name of each employee who works on all the projects controlled by department number 5 (use NOT EXISTS operator).

```
SELECT E.NAME
FROM EMPLOYEE E
WHERE NOT EXISTS (
```

```
    SELECT P.PNO
    FROM PROJECT P
    WHERE P.DNO = 5
    AND NOT EXISTS (
        SELECT *
        FROM WORKS_ON W
        WHERE W.PNO = P.PNO
        AND W.SSN = E.SSN
    )
);
```

NAME
Pallavi

PROGRAM 3

The commercial bank wants keep track of the customer's account information. Each customer may have any number of accounts and account can be shared by any number of customers. The system will keep track of the date of last transaction.

We store the following details.

a) Account: unique account-number, type and balance

b) Customer: unique customer-id, name and several addresses composed of street, city

and state

Perform the following operations on the database:

a. Create necessary tables and insert few tuples to all the relations.

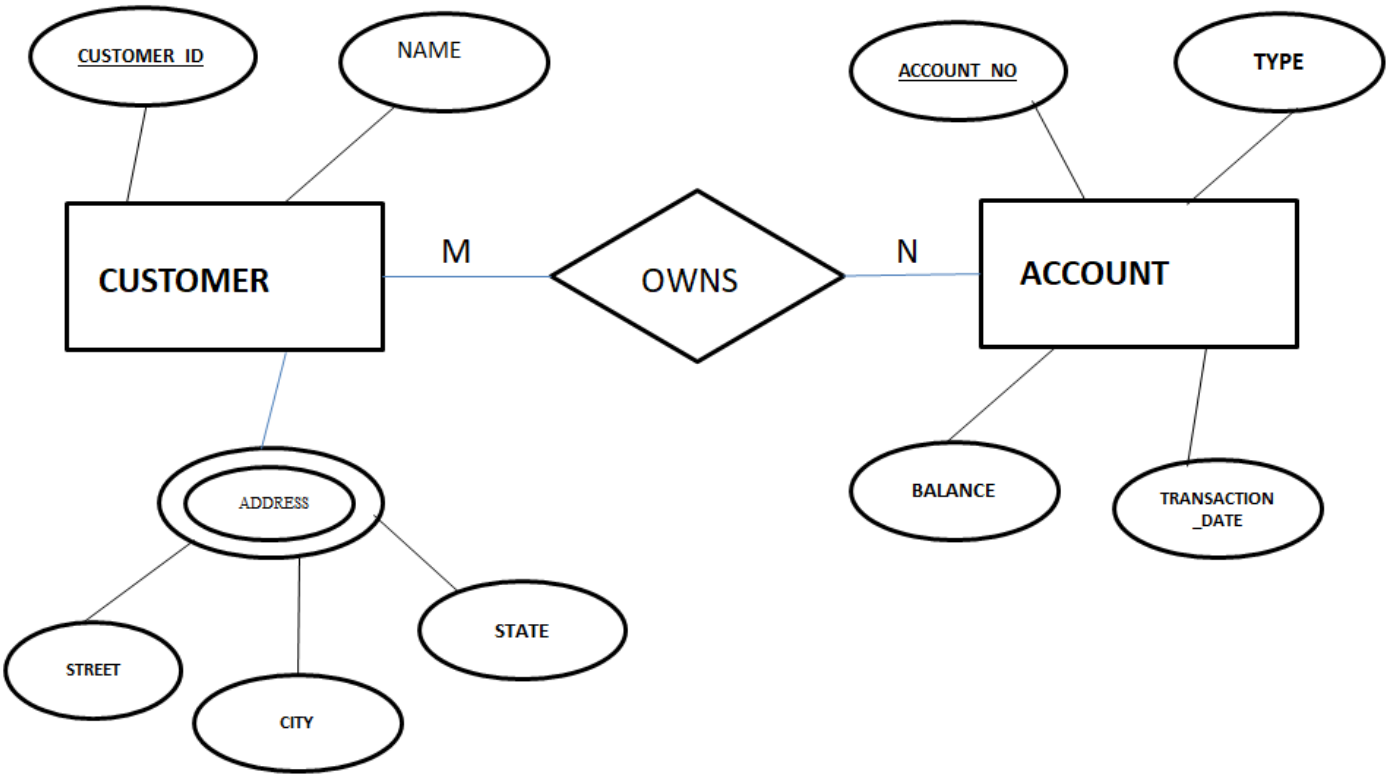
b. Add 5% interest to the customer who have less than 10000 balance.

c. List joint accounts involving more than three customers.

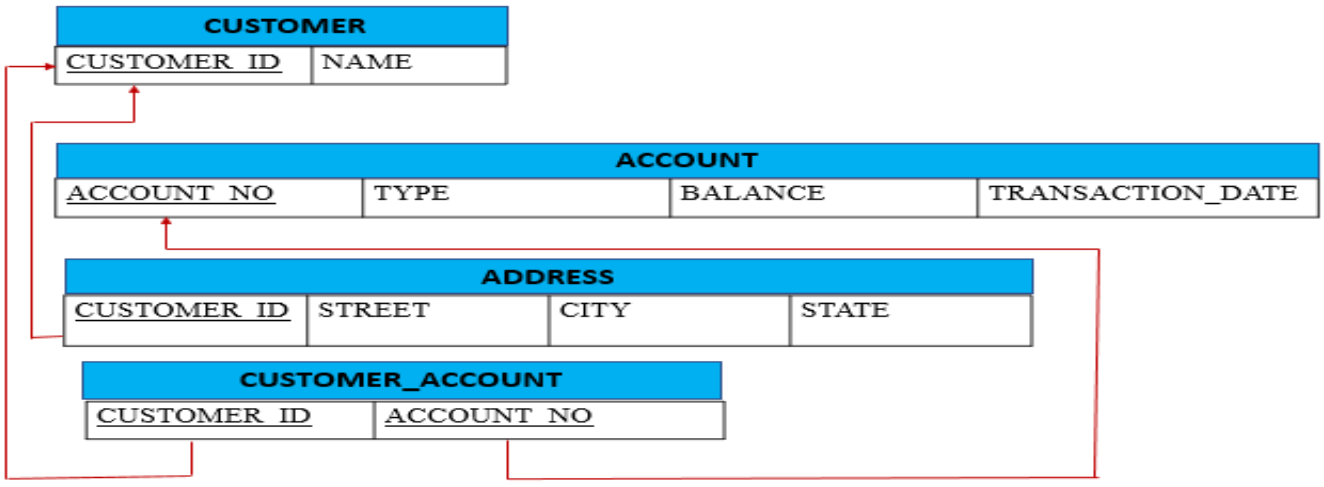
d. Find the total interest credited to each customer.

e. Find the customer who has not done any transaction.

ER DIAGRAM



SCHEMA DIAGRAM



```
CREATE DATABASE BANK_DB;
USE BANK_DB;
```

a. Create necessary tables and insert few tuples to all the relations.

#CUSTOMER Table

```
CREATE TABLE CUSTOMER
(
   CUSTOMER_ID INT PRIMARY KEY,
   NAME VARCHAR(50)
);
```

#ACCOUNT Table

```
CREATE TABLE ACCOUNT
(
   ACCOUNT_NO INT PRIMARY KEY,
   TYPE VARCHAR(20),
   BALANCE DECIMAL(10,2),
   TRANSACTION_DATE DATE
);
```

#ADDRESS Table

```
CREATE TABLE ADDRESS
(
```

```
    STREET VARCHAR(50),
    CITY VARCHAR(50),
    STATE VARCHAR(50),
    CUSTOMER_ID INT,
    PRIMARY KEY (CUSTOMER_ID, STREET),
    FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID)
        ON DELETE CASCADE
);
```

#CUSTOMER_ACCOUNT Table (Many-to-Many Relationship)

```
CREATE TABLE CUSTOMER_ACCOUNT
(
    CUSTOMER_ID INT,
    ACCOUNT_NO INT,
    PRIMARY KEY (CUSTOMER_ID, ACCOUNT_NO),
    FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (ACCOUNT_NO) REFERENCES ACCOUNT(ACCOUNT_NO)
        ON DELETE CASCADE
);
```

#Insert into CUSTOMER

CUSTOMER_ID	NAME
6	AMITH
1	JOHN
2	ARJUN
3	AJAY
4	JANE
5	SMITH

```
INSERT INTO CUSTOMER VALUES
(6,'AMITH'),
(1,'JOHN'),
(2,'ARJUN'),
(3,'AJAY'),
(4,'JANE'),
(5,'SMITH');
```

#Insert into ACCOUNT

ACCOUNT_NO	TYPE	BALANCE	TRANSACTION_DATE
107	SAVINGS	20000	
102	SAVINGS	5250	15-FEB-24
103	CURRENT	10000	10-MAR-24
104	CURRENT	10000	10-MAR-24
105	SAVINGS	8400	15-APR-24
106	CURRENT	19000	10-MAY-24

```
INSERT INTO ACCOUNT VALUES
(107,'Savings',20000,NULL),
(102,'Savings',5250,'2024-02-15'),
(103,'Current',10000,'2024-03-10'),
(104,'Current',10000,'2024-03-10'),
(105,'Savings',8400,'2024-04-15'),
(106,'Current',19000,'2024-05-10');
```

#Insert into ADDRESS

STREET	CITY	STATE	CUSTOMER_ID
CAR STREET	MANGALORE	KARNATAKA	6
OAK STREET	BANGALORE	KARNATAKA	1
RICHMOND CIRCLE	BANGALORE	KARNATAKA	2
MAIN STREET	MYSORE	KARNATAKA	3
BIRCH STREET	MYSORE	KARNATAKA	4
KULOOR	MANGALORE	KARNATAKA	5

INSERT INTO ADDRESS VALUES
('CAR STREET','Mangalore','Karnataka',6),
('OAK STREET','Bangalore','Karnataka',1),
('RICHMOND CIRCLE','BANGALORE','Karnataka',2),
('MAIN STREET','MYSORE','Karnataka',3),
('BIRCH STREET','MYSORE','Karnataka',4),
('KULOOR','Mangalore','Karnataka',5);

#Insert into CUSTOMER_ACCOUNT

CUSTOMER_ID	ACCOUNT_NO
1	102
1	103
2	103
3	103
4	105
5	103
6	107

INSERT INTO CUSTOMER_ACCOUNT VALUES
(1,102),
(1,103),
(2,103),
(3,103),
(4,105),
(5,103),

(6,107);

b) Add 5% Interest to Customers with Balance < 10000

SELECT * FROM ACCOUNT;

```
UPDATE ACCOUNT
SET BALANCE = BALANCE * 1.05
WHERE BALANCE < 10000;
```

SELECT * FROM ACCOUNT;

ACCOUNT_NO	TYPE	BALANCE	TRANSACTION_DATE
107	SAVINGS	20000	
102	SAVINGS	5512.5	15-FEB-24
103	CURRENT	10000	10-MAR-24
104	CURRENT	10000	10-MAR-24
105	SAVINGS	8820	15-APR-24
106	CURRENT	19000	10-MAY-24

c) List Joint Accounts Involving More Than Three Customers

```
SELECT ACCOUNT_NO, COUNT(CUSTOMER_ID) AS NUM_CUSTOMERS
FROM CUSTOMER_ACCOUNT
GROUP BY ACCOUNT_NO
HAVING COUNT(CUSTOMER_ID) > 3;
```

ACCOUNT_NO	NUM_CUSTOMERS
103	4

d) Find Total Interest Credited to Each Customer (Assuming interest = 5% of balance)

```
SELECT C.CUSTOMER_ID,
    C.NAME,
    SUM(A.BALANCE * 0.05) AS TOTAL_INTEREST
FROM CUSTOMER C
JOIN CUSTOMER_ACCOUNT CA ON C.CUSTOMER_ID = CA.CUSTOMER_ID
JOIN ACCOUNT A ON CA.ACCOUNT_NO = A.ACCOUNT_NO
GROUP BY C.CUSTOMER_ID, C.NAME;
```

CUSTOMER_ID	TOTAL_INTEREST
1	7625
2	5000
3	5000
4	4200
5	5000
6	10000

e) Find Customers Who Have Not Done Any Transaction

```
SELECT DISTINCT C.CUSTOMER_ID, C.NAME
FROM CUSTOMER C
JOIN CUSTOMER_ACCOUNT CA ON C.CUSTOMER_ID = CA.CUSTOMER_ID
JOIN ACCOUNT A ON CA.ACCOUNT_NO = A.ACCOUNT_NO
WHERE A.TRANSACTION_DATE IS NULL;
```

CUSTOMER_ID	NAME
6	AMITH

PROGRAM 4

A database is to be designed for a college to monitor students' progress throughout their course of study.The students are reading for a degree (such as B.E.) within the framework of the modular system. The college provides a number of Subjects (Modules), each being characterized by its code, title, credit value, module leader, teaching staff and the department they come from, prerequisite course. Department may be CSE, ISE etc. A Subject is co- ordinated by a module leader who shares teaching duties with one or more teachers. A Teacher may teach (and be a module leader for) more than one Subject. Students are free to choose any subject they wish. The database also contains some information about students including their Serial numbers, names, addresses, their past performance (i.e. subjects taken and Subject Examination Marks).

For this case study,

a. Analyze the data required, create the tables and insert the values.

b. Retrieve the Teacher names who are not Module leaders.

c. Display the department which offers the subject "Database Management System".

d. Display the number of Subjects taught by each Teacher.

e. Categorize students based on the following criterion: If Subject Examination Marks = 70

to 100 then CAT = 'Outstanding' If FinalIA = 40 to 69 then CAT = 'Average' If FinalIA<

39 then CAT = 'Weak'.

#Creation of Database

```
create database students;
use students;
```

 # a) Create Tables

#Teacher

```
CREATE TABLE Teacher(
    TeacherID INT PRIMARY KEY,
    TeacherName VARCHAR(50)
);
```

#Subject

```
CREATE TABLE Subject(
    SubjectCode VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(100),
    CreditValue INT,
    ModuleLeader VARCHAR(50),
    Department VARCHAR(10),
    PrerequisiteCourse VARCHAR(100),
    TeacherID INT,
    FOREIGN KEY (TeacherID) REFERENCES
Teacher(TeacherID)
);
```

#Teaches

```
CREATE TABLE Teaches(
```

```
    TeacherID INT,
    SubjectCode VARCHAR(10),
    PRIMARY KEY (TeacherID, SubjectCode),
    FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID),
    FOREIGN KEY (SubjectCode) REFERENCES Subject(SubjectCode)
);
```

#Student

```
CREATE TABLE Student(
    SerialNumber INT PRIMARY KEY,
    Name VARCHAR(50),
    Address VARCHAR(100)
);
```

#StudentProgress

```
CREATE TABLE StudentProgress(
    SerialNumber INT,
    SubjectCode VARCHAR(10),
    FinalIA INT,
    PRIMARY KEY (SerialNumber, SubjectCode),
    FOREIGN KEY (SerialNumber) REFERENCES Student(SerialNumber),
    FOREIGN KEY (SubjectCode) REFERENCES Subject(SubjectCode)
);
```

#Insert Values

#Insert into Teacher

TEACHERID	TEACHERNAME
1	Prof. Saranya
2	Prof.Chandramma
3	Prof Hemalatha
4	Prof Veena
5	Prof. Manjunath

INSERT INTO Teacher VALUES (1,'Prof. Saranya');
INSERT INTO Teacher VALUES (2,'Prof. Chandramma');
INSERT INTO Teacher VALUES (3,'Prof Hemalatha');
INSERT INTO Teacher VALUES (4,'Prof. Veena');
INSERT INTO Teacher VALUES (5,'Prof. Manjunath');

#Insert into Subject

SUBJECTCODE	TITLE	CREDITVALUE	MODULELEADER	DEPARTMENT	PREREQUISITECOURSE	TEACHERID
CML42	Database Management System	4	Prof.Saranya	CSE		1
CSE34	Operating Systems	3	Prof. Chandramma	ISE	Introduction to Computing	2
CSE33	Algorithm Design	4	Prof Hemalatha	CSE	Data Structures	3
ISE55	Network Security	3	Prof.Veena	ISE	Networking Fundamentals	4
ISE54	Advanced Python	3	Prof. Chandramma	ISE	Python Fundamentals	2
CSE12	C PROGRAMMING	4	Prof.Saranya	CSE		1

INSERT INTO Subject VALUES
('CML42','Database Management System',4,'Prof. Saranya','CSE',NULL,1),
('CSE34','Operating Systems',3,'Prof. Chandramma','ISE','Introduction to Computing',2),
('CSE33','Algorithm Design',4,'Prof Hemalatha','CSE','Data Structures',3),
('ISE55','Network Security',3,'Prof. Veena','ISE','Networking Fundamentals',4),
('ISE54','Advanced Python',3,'Prof. Chandramma','ISE','Python Fundamentals',2),

('CSE12','C Programming',4,'Prof. Saranya','CSE',NULL,1);

#Insert into Teaches

TEACHERID	SUBJECTCODE
1	CML42
1	CSE12
2	CSE34
3	CSE33
4	ISE55
5	ISE54

INSERT INTO Teaches VALUES (1,'CML42');
INSERT INTO Teaches VALUES (1,'CSE12');
INSERT INTO Teaches VALUES (2,'CSE34');
INSERT INTO Teaches VALUES (3,'CSE33');
INSERT INTO Teaches VALUES (4,'ISE55');
INSERT INTO Teaches VALUES (5,'ISE54');

#Insert into Student

SERIALNUMBER	NAME	ADDRESS
1001	John	RR NAGAR, BANGALORE
1002	Jane	JAYANAGAR, BANGALORE
1003	Adithya	Car street, MANGALORE
1004	NEHA	udayavara, UDUPI

INSERT INTO Student VALUES
(1001,'John','RR NAGAR, BANGALORE'),
(1002,'Jane','JAYANAGAR, BANGALORE'),
(1003,'Adithya','Car street, MANGALORE'),
(1004,'Neha','udayavara, UDUPI');

#Insert into StudentProgress

SERIALNUMBER	SUBJECTCODE	FINALIA
1001	CSE33	42
1001	CML42	68
1002	CSE33	70
1002	CSE34	65
1003	ISE55	38
1003	CML42	75
1004	ISE55	70
1004	CSE33	52

```
INSERT INTO StudentProgress VALUES
(1001,'CML42',85),
(1002,'CSE34',60),
(1003,'CSE33',35),
(1004,'ISE55',72);
```

b) Retrieve Teacher names who are NOT Module Leaders

```
SELECT TeacherName
FROM Teacher
WHERE TeacherName NOT IN
(
SELECT ModuleLeader
FROM Subject
);
```

TEACHERNAME
Prof. Manjunath

c) Display department offering Database Management System

```
SELECT Department
FROM Subject
```

WHERE Title = 'Database Management System';

DEPARTMENT
CSE

d) Display number of subjects taught by each teacher

```
SELECT T.TeacherName, COUNT(Tc.SubjectCode) AS NumberOfSubjects
FROM Teacher T
JOIN Teaches Tc
ON T.TeacherID = Tc.TeacherID
GROUP BY T.TeacherName;
```

TEACHERNAME	NUMBEROFSUBJECTS
Prof Hemalatha	1
Prof Veena	1
Prof. Manjunath	1
Prof. Saranya	2
Prof.Chandramma	1

e) Categorize Students based on Marks

```
SELECT S.SerialNumber,
    S.Name,
    SP.SubjectCode,
    SP.FinalIA,
    CASE
        WHEN SP.FinalIA BETWEEN 70 AND 100 THEN 'Outstanding'
        WHEN SP.FinalIA BETWEEN 40 AND 69 THEN 'Average'
        ELSE 'Weak'
    END AS Category
FROM Student S
```

JOIN StudentProgress SP

ON S.SerialNumber = SP.SerialNumber;

SERIALNUMBER	NAME	SUBJECTCODE	CATEGORY
1001	John	CSE33	Average
1001	John	CML42	Average
1002	Jane	CSE33	Outstanding
1002	Jane	CSE34	Average
1003	Adithya	ISE55	Weak
1003	Adithya	CML42	Outstanding
1004	NEHA	ISE55	Outstanding
1004	NEHA	CSE33	Average